

TaskQuest

Gestionnaire de tâches gamifié

Projet individuel — Java — Bachelor Informatique B1 — S2 2025-2026

Paramètre	Valeur
Filière	Bachelor Informatique — Option Informatique
Niveau	B1 — Semestre 2
Module	Programmation Ytrack
Volume horaire	28h encadrées
Période	21 mars → 7 juin 2026
Travail	Individuel
Rendu	GitHub — dépôt public
ECTS	3 crédits

1. Présentation du projet

Vous devez réaliser TaskQuest, une application de bureau en Java permettant à un utilisateur de gérer ses tâches sous forme de quêtes RPG. Chaque tâche accomplie rapporte de l'expérience (XP) à un personnage qui monte de niveau et débloque des titres au fil de sa progression.

L'objectif pédagogique est de mobiliser l'ensemble des compétences Java du semestre dans un projet ludique et cohérent : programmation orientée objet, gestion des exceptions, persistance des données, interface graphique et bonnes pratiques de développement.

Contexte narratif

L'utilisateur incarne un aventurier dont chaque tâche du quotidien est une quête. Terminer une quête rapporte de l'XP. Accumuler de l'XP fait monter de niveau. Monter de niveau débloque un titre (Novice, Apprenti, Développeur, Vétéran, Architecte, Légende).

2. Choix technologiques

2.1 Interface graphique

Vous devez implémenter une interface graphique. Deux options sont disponibles, chacune avec son niveau de difficulté :

Technologie	Difficulté	Points forts	Points d'attention
Swing	★ ★ Intermédiaire	Intégré au JDK, aucune dépendance, nombreux tutoriels	API verbeuse, look & feel daté, layouts parfois complexes
JavaFX	★ ★ ★ Avancé	Moderne, CSS natif, FXML possible, rendu propre	Dépendance externe à configurer (OpenJFX), courbe d'apprentissage plus haute

Recommandation

Si c'est votre première interface graphique en Java, commencez par Swing. JavaFX est valorisé mais ne sera pas mieux noté si le reste du projet est bâclé. La qualité du code prime sur la technologie choisie.

2.2 Persistance des données

Vous devez persister les données de l'application entre deux lancements. Deux formats sont acceptés :

Format	Difficulté	Points forts	Points d'attention
JSON (fichier)	★ ★ Intermédiaire	Simple, lisible, facile à déboguer, aucune config	Gestion manuelle du parsing, risque de corruption
SQLite (base de données)	★ ★ ★ Avancé	Requêtes structurées, robuste, scalable	Driver JDBC à configurer, SQL à maîtriser

Important — Security by design

Quel que soit le format choisi, vous devez valider et assainir toutes les données avant persistance. Pour SQLite : utilisez impérativement des requêtes préparées (PreparedStatement) — jamais de concaténation de chaînes dans les requêtes SQL.

2.3 Bibliothèques externes

L'application doit fonctionner sans bibliothèque externe. Vous pouvez en ajouter uniquement pour des fonctionnalités bonus (ex. : Gson pour le JSON, Jackson, SQLite JDBC driver). Ces ajouts ne compensent pas un code de mauvaise qualité et ne sont pas notés en eux-mêmes — seul leur usage pertinent peut apporter des points bonus.

3. Architecture obligatoire

Votre projet doit respecter une architecture MVC (Modèle - Vue - Contrôleur). Chaque couche doit être dans un package Java distinct.

Package	Responsabilité	Exemples de classes
model	Logique métier et données	Player, Quest, DailyQuest, Reward, QuestStatus
view	Interface graphique uniquement	MainWindow, QuestPanel, PlayerPanel, DialogForm
controller	Coordination entre modèle et vue	QuestController, PlayerController
repository	Accès aux données (JSON ou SQL)	QuestRepository, PlayerRepository
exception	Exceptions métier personnalisées	InvalidQuestException, DataCorruptedException

Règle stricte

Un composant de la couche view ne doit jamais accéder directement au repository. Toute action passe par le controller. Toute lecture ou écriture de données passe par le repository.

3.1 Modèle objet minimal

Votre modèle doit inclure au minimum les éléments suivants. Vous pouvez enrichir librement.

- Quest (classe abstraite) : title, description, xpReward, status (enum QuestStatus)
- DailyQuest extends Quest : récurrente, se réinitialise chaque jour
- OneTimeQuest extends Quest : quête à accomplir une seule fois
- Player : name, level, currentXP, totalXP, title — méthode addXP(int) avec logique de montée de niveau
- Reward : libellé du titre débloqué à chaque palier de niveau
- QuestStatus (enum) : TODO, IN_PROGRESS, DONE

4. Fonctionnalités attendues

4.1 Fonctionnalités obligatoires

L'application doit permettre au minimum :

1. Créer une quête (titre, description, XP récompense, type Daily ou OneTime)
2. Afficher la liste des quêtes avec leur statut
3. Marquer une quête comme terminée (attribution automatique de l'XP au joueur)
4. Supprimer une quête

5. Afficher le profil du joueur : nom, niveau, XP actuelle, titre actuel, barre de progression
6. Persister et recharger toutes les données au lancement et à la fermeture de l'app

4.2 Fonctionnalités bonus (non obligatoires)

Ces fonctionnalités peuvent apporter des points supplémentaires si elles sont bien implémentées :

- Filtrage et tri des quêtes (par statut, par XP, par type)
- Historique des quêtes accomplies avec date et heure
- Système de catégories de quêtes (Travail, Personnel, Apprentissage...)
- Export de l'historique en fichier texte ou CSV
- Fenêtre de statistiques (XP gagnée par jour, taux de complétion...)
- Thème visuel personnalisé (mode sombre, couleurs)

5. Gestion des exceptions et des erreurs

La robustesse de l'application est un critère de notation à part entière.

5.1 Règles obligatoires

- Toutes les entrées utilisateur doivent être validées avant traitement (titre vide, XP négative, champ manquant...)
- Les erreurs de lecture/écriture (fichier absent, JSON malformé, erreur SQL) doivent être catchées et affichées à l'utilisateur via un message clair dans l'interface — jamais via un simple `System.out`
- Créer au moins deux exceptions métier personnalisées (ex. `InvalidQuestException`, `PlayerNotFoundException`)
- Ne jamais attraper `Exception` ou `Throwable` de façon générique sans traitement adapté

5.2 Security by design — règles minimales

- Validation côté code de toutes les données avant tout traitement ou persistance
- Longueur maximale des champs titre et description
- XP récompense : entier strictement positif, borné à une valeur maximale raisonnable
- Aucune construction de requête SQL par concaténation (`PreparedStatement` obligatoire si `SQLite`)
- Fichier de données : vérifier son existence et son intégrité au démarrage, gérer le cas d'un fichier corrompu sans planter l'application

6. Bonnes pratiques de code

6.1 Nommage et structure

- Classes : `PascalCase` (`QuestController`, `PlayerRepository`)
- Méthodes et variables : `camelCase` (`addXP`, `currentLevel`)
- Constantes : `UPPER_SNAKE_CASE` (`MAX_LEVEL`, `XP_PER_LEVEL`)
- Packages : tout en minuscules (`model`, `view`, `controller`)
- Un fichier = une classe publique

6.2 Documentation

- Chaque classe publique doit avoir un commentaire Javadoc décrivant son rôle
- Chaque méthode publique doit avoir un commentaire Javadoc avec `@param`, `@return` et `@throws` si applicable
- Un fichier `README.md` à la racine du dépôt décrivant : le projet, comment le lancer, les dépendances éventuelles, les choix techniques

6.3 Git

- Ajouter Hénan à votre repo
- Nombre de commits approprié selon les bonnes pratiques
- Les messages de commit doivent être clairs et en français ou anglais (pas de 'fix', 'test', 'aaa')
- Ne pas commit le dossier de build (target/ ou out/) ni les fichiers IDE (.idea/, *.iml, .classpath...) — utiliser un .gitignore adapté

7. Critères de notation

Critère	Sous-critères évalués	Poids
Respect de la POO	Héritage, polymorphisme, encapsulation, abstractions cohérentes	20 %
Architecture MVC	Séparation des couches, packages, responsabilités claires	15 %
Gestion des exceptions et I/O	Exceptions métier, robustesse, messages utilisateur, I/O sans crash	15 %
Security by design	Validation inputs, pas d'injection SQL, intégrité des données	10 %
Interface graphique	Fonctionnelle, claire, utilisable sans connaissance du code	15 %
Bonnes pratiques de code	Nommage, lisibilité, structure, pas de code mort	10 %
Gestion Git	Commits réguliers, messages clairs, .gitignore, README	10 %
Documentation Javadoc	Classes et méthodes publiques documentées, README complet	5 %
Bonus (fonctionnalités avancées)	Cf. section 4.2 — appréciation qualitative	+ 5 pts max

8. Livrables

Un seul livrable est attendu : l'URL de votre dépôt GitHub public, communiquée à votre intervenant avant la date limite.

Le dépôt doit contenir au minimum :

- Le code source complet du projet
- Un fichier README.md à la racine (voir section 6.2)
- Un fichier .gitignore approprié
- La documentation Javadoc générée (dossier doc/)

Date limite de rendu

À définir par votre intervenant. Tout commit effectué après la date limite ne sera pas pris en compte dans la notation.

9. Conseils pour réussir

7. Commencez par le modèle objet — Player, Quest, QuestStatus — avant de toucher à l'interface graphique.
8. Faites fonctionner la persistance très tôt : un bug de sauvegarde découvert en fin de projet est catastrophique.
9. Committez souvent : après chaque fonctionnalité qui marche, faites un commit. Cela sécurise votre travail.
1. Ne sacrifiez pas la qualité du code pour ajouter des bonus — un projet simple mais propre est toujours mieux noté qu'un projet riche mais illisible.
2. Testez les cas limites : que se passe-t-il si le fichier de données est absent au démarrage ? Si l'utilisateur laisse un champ vide ?
3. Lisez le README.md de votre propre projet avant de rendre : si quelqu'un peut lancer l'application en le suivant, c'est bon signe.

Bonne quête !

Ynov Campus Rennes · Bachelor Informatique B1 · 2025-2026